

Emergency Shelter Allocation Platform: A Capacity-Aware Web-Based Decision-Support System for Disaster Shelter Recommendation

Sharan Poojari, Suhas S Kattimani, and Harshil Gurjar

Abstract—During disasters, affected people struggle to identify shelters that match their location, required facilities and accessibility needs, while administrators lack a consolidated, up-to-date view of shelter capacity. The result is search delay, longer travel, overcrowding at some shelters and under-use of others. This paper presents the design of the Emergency Shelter Allocation Platform, a web-based decision-support system that centralizes shelter information (location, capacity, occupancy, facilities and status) and recommends suitable shelters to users. The platform applies a staged allocation pipeline of availability filtering, capacity feasibility, suitability checking, distance computation and weighted ranking, and returns the best feasible shelters together with route and capacity details. Administrators register shelters and update occupancy, after which recommendations are recalculated so that later users are steered away from shelters approaching capacity. We describe the layered architecture (React front end, Node.js/Express REST API, PostgreSQL database, OpenStreetMap/Leaflet mapping and a routing service), the ranking model, and a planned evaluation against a nearest-shelter baseline using travel distance, overload ratio and utilization balance. A review of eleven related works positions the platform relative to existing research. The system is under development, and experimental results will be reported in a later revision.

Index Terms—Disaster management, emergency shelter allocation, decision support system, capacity-aware allocation, geospatial routing, web application, multi-criteria ranking.

I. INTRODUCTION

DISASTERS such as floods, cyclones and earthquakes force large numbers of people to leave their homes at short notice, and public buildings such as schools, community halls and relief camps are pressed into service as emergency shelters. In such moments an affected person must decide where to go with very little reliable information: which shelters are open, how much space remains, whether they provide the facilities the person needs, and how long the journey will take.

Without a shared and current view of shelter status, these decisions tend to be made on familiarity or proximity. This can send many people to the same nearby shelter while others remain under-used, lengthening travel and evacuation time and exposing shelters to overcrowding. Assigning people to the nearest shelter is a common practice, but it ignores capacity and can cause load imbalance and slow evacuation [1].

Draft prepared for the Mini Project (CSE7102), Project ID PRJ_465, School of Computer Science and Engineering, Presidency University.

The authors are with the School of Computer Science and Engineering, Presidency University, Bengaluru, India. The project is carried out under the supervision of Irfan Rajab Bhatt, Associate Professor.

Emergency administrators face the mirror-image problem: without centralized shelter information it is difficult to monitor occupancy, redirect people, or plan supplies.

The key problems addressed in this work are:

- 1) difficulty in finding suitable shelters quickly;
- 2) risk of shelter overcrowding;
- 3) longer travel and evacuation time;
- 4) lack of centralized shelter information;
- 5) inefficient utilization of shelter capacity; and
- 6) difficulty in considering accessibility and facilities.

To address them we present the *Emergency Shelter Allocation Platform*, a software-based web application that helps affected people and emergency administrators identify suitable shelters during disasters. The platform uses shelter information such as location, capacity, occupancy, facilities and current status to support faster and more efficient allocation, filters and ranks the available shelters against user requirements, and presents clear recommendations. Its objectives are (i) faster identification of suitable shelters, (ii) capacity-aware shelter allocation, (iii) reduction of shelter overcrowding, (iv) centralized and updated shelter information, and (v) clear decision support for emergency response.

The contributions of this paper are:

- a requirements analysis that links the six key problems to concrete platform features (Section III);
- a layered web architecture and data model for centralized shelter information (Section IV);
- a staged, capacity-aware allocation and ranking model with an occupancy feedback loop (Section V);
- an implementation plan based on open-source technologies and an evaluation plan against a nearest-shelter baseline (Sections VI and VII); and
- a review of eleven related works that positions the platform in the literature (Section II).

This paper describes the system at the design stage; implementation and evaluation are in progress.¹ The remainder of the paper is organized as follows. Section II reviews related work, Section III analyzes the problem and requirements, Section IV presents the architecture, Section V describes the allocation methodology, Section VI covers implementation, Section VII sets out the evaluation plan, Section VIII discusses limitations, and Section IX concludes.

¹Project repository:

<https://github.com/HarshilxAI/Emergency-Shelter-Allocation-Platform>

II. RELATED WORK

We reviewed eleven works on shelter allotment, emergency facility location and disaster resource allocation. They are summarized in Table I and discussed below.

A. Shelter Allotment and Evacuation Planning

Yang *et al.* [1] formalize intelligent shelter allotment as the assignment of routes, exits and shelters to evacuee groups for quick and safe evacuation. They contrast the nearest exit or shelter (NES) practice, which avoids some movement conflicts but ignores transportation choke-points and can overload shelters, with capacity-constrained route planning (CCRP), which speeds evacuation but does not separate groups heading to different shelters. Their CARES approach detects spatial anomalies, meaning groups whose allotted shelter differs from that of their neighbors, and re-allots them. In a Hajj case study in Makkah it provides much faster evacuation than NES and much lower movement conflict than CCRP. Our platform shares the capacity-aware premise of [1] but targets individual and household requests through a web interface rather than planner-level, group-scale route assignment.

B. Resource Allocation Under Time and Cost Constraints

Zhang *et al.* [2] propose a multiagent-based decentralised approach, using domain transportation theory, to allocate resources for multi-task emergency events under time and cost constraints. Wang *et al.* [3] model multi-period post-disaster allocation and solve it with a multi-objective cellular genetic algorithm. Both show the value of explicit constraints when allocating scarce resources, which motivates the hard capacity and status constraints in our pipeline.

C. Data-Driven and Crowdsourced Decision Support

Amiran *et al.* [4] apply machine learning to site selection for earthquake evacuation and to neighborhood navigation. Sarma *et al.* [5] combine crowdsourced information with multi-criteria decision making (MCDM) in a cost-minimization model for disaster relief. The MCDM idea underlies our weighted ranking, and crowdsourced or learned inputs are natural future extensions.

D. Emergency Facility Location

Li *et al.* [6] give a model and algorithms for the emergency service facility location problem. Tanti *et al.* [7] extend facility location to the post-disaster setting with uncertain demand, capacity, cost and travel time. These works decide *where* facilities should be placed. The platform instead assumes that shelters exist and recommends among them at request time, using the same quantities (capacity and travel time) that [7] identifies as important.

E. Post-Disaster Material Allocation and Multi-Period Planning

Cui *et al.* [8] study emergency material allocation when the disaster disrupts supply chains and puts routes at risk, optimizing transport time and cost. Rodríguez-Espíndola [9] jointly considers facility location, resource allocation and distribution across several periods of simultaneous disasters. They highlight route risk and time-varying conditions, which we note as extensions of the routing and occupancy components.

F. Toward Practical Decision Support

Rajalakshmi *et al.* [10] discuss AI-supported demand analysis, logistics and resource allocation in humanitarian aid. Closest to shelter-level allocation, Basu *et al.* [11] define a utility function that quantifies how urgently each shelter needs each resource, collect needs over a smartphone-based delay-tolerant network when communication infrastructure is down, and allocate resources with an integer program that minimizes resource deficit and deployment time. That work allocates supplies to shelters, whereas our platform allocates people to shelters, but its modelling of urgency, time and capacity, and its attention to low-connectivity data collection, carry over.

G. Positioning

In the summaries and abstracts we reviewed, the contributions are mainly models and algorithms for planners. We did not find one presented as a deployable, end-user-facing web service that combines centralized shelter data, transparent multi-criteria ranking, route guidance and administrator-controlled occupancy updates. The Emergency Shelter Allocation Platform targets this gap, deliberately using simple, explainable logic that emergency staff can inspect and adjust.

III. PROBLEM ANALYSIS AND REQUIREMENTS

A. From Key Problems to Platform Features

Table II maps each key problem to the platform feature intended to address it.

B. Functional Requirements

The expected outputs of the platform are the following.

- **FR1, Shelter discovery:** display shelters that are currently available and relevant to the user's location.
- **FR2, Smart ranking:** rank eligible shelters using distance, remaining capacity, suitability and facility criteria.
- **FR3, Allocation result:** show the recommended shelter with its available capacity, occupancy, facilities and contact details.
- **FR4, Route guidance:** present the recommended route and its distance or time through the map interface.
- **FR5, Administrator view:** provide shelter registration, status and occupancy updates, and allocation monitoring.
- **FR6, Decision support:** reduce manual searching and help distribute users across suitable shelters instead of overloading a single location.

TABLE I
SUMMARY OF REVIEWED WORKS AND THEIR RELEVANCE TO THE PLATFORM

Ref.	Focus	Approach as reported	Relevance to the platform
[1]	Shelter allotment for evacuation	CARES assigns routes, exits and shelters and avoids spatial anomalies; compared with NES and CCRP in a Hajj case study	Motivates capacity-aware allocation; shows why nearest-shelter assignment can overload shelters
[2]	Emergency resource allocation	Decentralised multiagent allocation using domain transportation theory under time and cost constraints	Supports constraint-aware allocation; informs a future distributed extension
[3]	Multi-period post-disaster allocation	Multi-objective cellular genetic algorithm for uncertain, multi-period relief allocation and routing	Basis for handling time-varying occupancy
[4]	Evacuation-center site selection	Machine learning for site selection and neighborhood navigation in earthquakes	Motivates learned suitability and demand estimates
[5]	Disaster-relief resource allocation	Cost minimization with crowdsourced information and MCDM	Supports multi-criteria ranking and crowdsourced updates
[6]	Emergency facility location	Model and algorithms for locating emergency service facilities	Foundation for location-based shelter matching
[7]	Dynamic facility location	Optimization under uncertain demand, capacity, cost and travel time	Relevant to dynamic capacity and travel-time handling
[8]	Emergency material allocation	Considers disrupted supply chains and route risk; optimizes transport time and cost	Motivates route-risk awareness
[9]	Multi-period disaster management	Joint facility location, resource allocation and distribution across periods and simultaneous disasters	Relevant to multi-disaster suitability and future extensions
[10]	AI for humanitarian logistics	AI-supported demand analysis, logistics and resource allocation	Motivates AI-assisted decision support
[11]	Resource allocation to shelters	Utility function for resource urgency; DTN-based knowledge sharing; integer program minimizing deficit and deployment time	Shelter-level needs modelling; low-connectivity data collection

TABLE II
KEY PROBLEMS AND PLATFORM RESPONSE

Key problem	Platform response
Difficulty finding suitable shelters quickly	Location-aware shelter discovery with filtering by user requirements
Risk of overcrowding	Remaining-capacity check before every recommendation; occupancy updated after each allocation
Longer travel and evacuation time	Distance or route time used in ranking; route shown on the map
Lack of centralized shelter information	Single database of shelter data maintained by administrators
Inefficient use of shelter capacity	Capacity term in the ranking spreads users across suitable shelters
Accessibility and facilities hard to consider	Facility and accessibility attributes used as constraints and scoring criteria

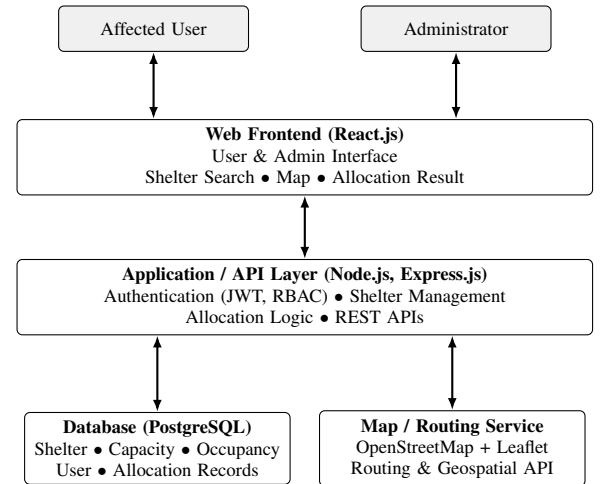


Fig. 1. Layered architecture of the Emergency Shelter Allocation Platform. The API layer returns a ranked shelter list with route and capacity details to the interface.

C. Design Considerations and Scope

The platform is designed as a decision-support tool: final emergency decisions remain with authorized personnel. Beyond the functional requirements, we adopt the following design considerations: a responsive browser interface usable on phones and desktops; token-based authentication with role-based access control; timestamps on shelter data so that stale information can be identified; a modular REST interface so that other systems can be integrated later; and reliance on open-source components so that no dedicated hardware is required.

IV. SYSTEM ARCHITECTURE

A. Layered Architecture

Fig. 1 shows the architecture. Affected users and administrators interact with a web front end. The front end calls an application/API layer that holds authentication, shelter management and the allocation logic. The API layer reads and writes the database and queries a routing and geospatial service. The end-to-end flow is: user or administrator → web interface → API and allocation logic → database and routing service → ranked shelter and route.

TABLE III
MAIN ENTITIES OF THE PROPOSED DATA MODEL

Entity	Key attributes
Shelter	identifier, name, coordinates, address, contact, capacity, occupancy, status, facilities (e.g., medical aid, water, sanitation, power), accessibility rating, suitable disaster types, last-updated time
User	identifier, role (affected user or administrator), credentials, contact
Allocation record	identifier, user, shelter, group size, request location, time, state
Request (transient)	location, group size, required and preferred facilities, disaster type

B. Components

1) *Web Frontend*: A React.js single-page interface (HTML5, CSS3, responsive layout) provides shelter search, a map view and the allocation result for affected users, and shelter registration, occupancy update and allocation monitoring screens for administrators.

2) *Application/API Layer*: A Node.js service using Express.js exposes REST endpoints for authentication, shelter management and allocation. The allocation logic of Section V runs in this layer, so that ranking rules stay in one place and can be adjusted without changing the interface.

3) *Database*: PostgreSQL stores shelter, capacity, occupancy, user and allocation data. It is ready to be extended with PostGIS [14] for spatial indexing and distance queries.

4) *Map and Routing Service*: Maps are rendered with Leaflet [13] on OpenStreetMap [12] data. A routing API or service provides road distance and travel time between the user and candidate shelters, and the recommended route is drawn on the map.

C. Data Model

Table III lists the main entities and attributes of the proposed schema.

D. Security and Privacy

Access is controlled with JSON Web Tokens [15] and role-based access control: only administrators can register shelters or change status and occupancy, while affected users can query and request allocations. To limit exposure of personal data the platform stores only the attributes needed for allocation and does not require identity beyond a contact detail. Traffic is to be served over HTTPS.

V. ALLOCATION METHODOLOGY

A. Functional Workflow

The workflow of Fig. 2 has seven steps. The user or administrator logs in or accesses the interface; the user enters a location and requirements; the system fetches shelters and their current status; filters them by capacity and suitability; ranks the remainder by distance, capacity and suitability; and displays the shelter with route and details. Finally the

allocation is recorded and the shelter's occupancy is updated. When an administrator later updates capacity or occupancy, recommendations are recalculated.

B. Allocation Model

Let S be the set of shelters. Shelter $s \in S$ has location ℓ_s , capacity C_s , occupancy O_s , status $\sigma_s \in \{\text{open}, \text{full}, \text{closed}\}$, a facility set F_s , a set Δ_s of disaster types for which it is suitable, and an accessibility rating $\alpha_s \in [0, 1]$. A request r has location ℓ_r , group size n_r , required facilities F_r^{req} , preferred facilities F_r^{pref} and disaster type δ_r . The remaining capacity is

$$R_s = C_s - O_s. \quad (1)$$

Feasibility. Unavailable, full or unsuitable shelters are removed, leaving the feasible set

$$S_r = \{s \in S : \sigma_s = \text{open}, R_s \geq n_r, F_r^{\text{req}} \subseteq F_s, \delta_r \in \Delta_s\}. \quad (2)$$

The condition $\delta_r \in \Delta_s$ captures disaster-specific suitability, for example excluding a low-lying shelter during a flood.

Distance. For $s \in S_r$, let d_{rs} be the road distance (or travel time) from ℓ_r to ℓ_s returned by the routing service, normalized by

$$\hat{d}_{rs} = \frac{d_{rs}}{\max_{s' \in S_r} d_{rs'}}. \quad (3)$$

Ranking score. Each feasible shelter receives a weighted score

$$\text{Score}(s | r) = w_d (1 - \hat{d}_{rs}) + w_c \kappa_{rs} + w_f \varphi_{rs} + w_a \alpha_s, \quad (4)$$

with $w_d, w_c, w_f, w_a \geq 0$ and $w_d + w_c + w_f + w_a = 1$. The spare-capacity term

$$\kappa_{rs} = \frac{R_s - n_r}{C_s} \quad (5)$$

is the fraction of capacity that would remain after the allocation, and the facility-match term

$$\varphi_{rs} = \frac{|F_r^{\text{pref}} \cap F_s|}{|F_r^{\text{pref}}|} \quad (6)$$

is set to 1 when $F_r^{\text{pref}} = \emptyset$. The weights are configurable. Initial values will be set by the project team and tuned through the sensitivity analysis described in Section VII.

Because κ_{rs} rewards shelters that keep spare capacity, and occupancy is updated after each allocation, later requests are steered toward other suitable shelters. This is how the platform aims to distribute users instead of overloading a single location. The nearest-shelter baseline of [1] corresponds to choosing $\arg \min_s d_{rs}$ without the capacity and suitability checks.

C. Algorithm

Algorithm 1 summarizes the procedure. To limit calls to the routing service, shelters in S_r are first ordered by great-circle distance and road distances are requested only for the nearest M of them.

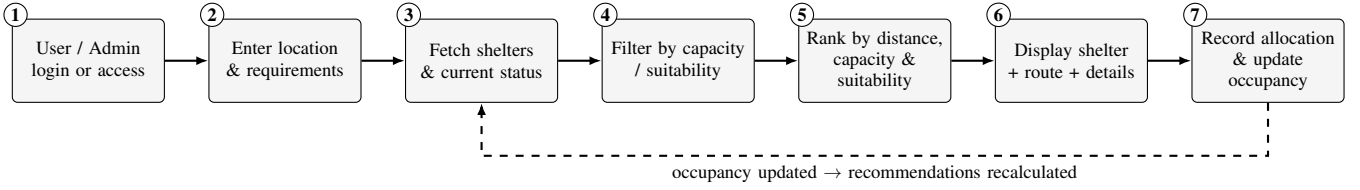


Fig. 2. Proposed functional workflow. The dashed loop shows that every recorded allocation or administrator update changes shelter occupancy and triggers recalculation.

Algorithm 1 Capacity-Aware Shelter Ranking

Input: request r , shelter set S , weights $\mathbf{w}$, list size k , candidate limit M

Output: ranked list L of at most k shelters

```

1:  $S_r \leftarrow \{s \in S : \sigma_s = \text{open} \wedge R_s \geq n_r\}$ 
2:  $S_r \leftarrow \{s \in S_r : F_r^{\text{req}} \subseteq F_s \wedge \delta_r \in \Delta_s\}$ 
3: if  $S_r = \emptyset$  then
4:   return  $\emptyset$  and flag the request for administrators
5: end if
6: keep the  $M$  shelters of  $S_r$  nearest to  $\ell_r$  by great-circle distance
7: query the routing service for  $d_{rs}$  of each kept shelter
8: for each kept shelter  $s$  do
9:   compute  $\hat{d}_{rs}$ ,  $\kappa_{rs}$ ,  $\phi_{rs}$  and  $\text{Score}(s | r)$ 
10: end for
11: sort by score in descending order
12: return the first  $k$  shelters with route, capacity and contact details
  
```

D. Occupancy Update and Recalculation

When a user confirms a recommendation, an allocation record is created and O_s is increased by n_r in a single database transaction that first re-checks $R_s \geq n_r$ under a row-level lock. This prevents two simultaneous requests from taking the last places in the same shelter. When an administrator changes capacity, status or occupancy, the next request sees the new values, and open recommendation screens are refreshed. Each shelter record carries a last-updated time so that stale entries can be flagged.

E. Complexity

Filtering is linear in $|S|$. Ordering by great-circle distance costs $O(|S_r| \log |S_r|)$, after which at most M routing queries are made, with $M \ll |S|$. With a spatial index the initial candidate search can be restricted to a radius around ℓ_r .

VI. IMPLEMENTATION PLAN

A. Technology Stack

Table IV lists the technologies selected for each layer. No dedicated hardware is required: development and testing run on a standard laptop or desktop with internet access.

B. Data Sources and Project Status

Shelter locations and road networks come from OpenStreetMap. The project plan also identifies Bhuvan, the India Meteorological Department, OpenStreetMap Overpass and

TABLE IV
TECHNOLOGY STACK

Layer	Technology
Frontend	React.js, HTML5, CSS3, responsive UI
Backend	Node.js with Express.js, REST APIs
Database	PostgreSQL (PostGIS-ready)
Maps and routing	OpenStreetMap, Leaflet, routing API/service
Authentication	JWT, role-based access control
Development tools	Visual Studio Code, Git, GitHub, npm
Testing	Postman, browser developer tools
Deployment	GitHub-based workflow, cloud hosting platform

OpenRouteService as candidate data and API sources, which are being evaluated. At the time of writing, the literature review, architecture design and output planning are complete, and the repository contains the project setup, planning documents, architecture and implementation plan, and the evaluation of APIs and open-source resources. Development is scheduled from August to December 2026, with checkpoints at 30% and 80% completion, a poster exposition and final report submission.

VII. EVALUATION PLAN AND EXPECTED RESULTS

This section defines how the platform will be evaluated. No experimental results are reported yet.

A. Scenarios and Baselines

We will build synthetic scenarios of varying size, with different numbers of shelters, requests and initial occupancy levels, using shelter candidates derived from OpenStreetMap public facilities. We also plan to backtest against a historical flood event; the project plan targets a flood in Assam, India. The proposed method will be compared with (a) the nearest-shelter baseline of [1] and (b) a capacity-checked nearest-shelter baseline that applies only the capacity filter and ranks by distance. Each configuration will be repeated with different random seeds and reported as mean and standard deviation.

B. Metrics

Table V defines the metrics. Utilization balance uses Jain's fairness index [16] over occupancy ratios $\rho_s = O_s/C_s$.

TABLE V
EVALUATION METRICS

Metric	Definition	Purpose
Mean travel distance/time	Mean d_{rs} between requests and allocated shelters	Travel and evacuation time
Overload ratio	$\frac{1}{ S } \sum_s \max(0, O_s - C_s) / C_s$	Overcrowding (target 0)
Utilization balance	$J = \frac{(\sum_s \rho_s)^2}{ S \sum_s \rho_s^2}$	Even use of capacity
Feasibility rate	Share of requests with at least one feasible shelter	Coverage
Allocation latency	95th-percentile response time of the ranking API	Suitability for time-critical use
Rank stability	Kendall's τ between rankings under perturbed weights	Sensitivity to weights

TABLE VI
COMPARISON WITH BASELINES (TO BE COMPLETED)

Method	Dist.	Overload	J	Latency
Nearest shelter [1]	TBD	TBD	TBD	TBD
Capacity-checked nearest	TBD	TBD	TBD	TBD
Proposed ranking	TBD	TBD	TBD	TBD

C. Sensitivity Analysis

We will vary the weights (w_d, w_c, w_f, w_a) over a grid and report how mean travel distance, overload ratio and utilization balance change, to expose the trade-off between short trips and even use of capacity and to choose default weights.

D. Results

Table VI is the template for the main comparison. *[Draft placeholder: values will be inserted after implementation and experiments.]*

VIII. LIMITATIONS AND ETHICAL CONSIDERATIONS

Data freshness. Recommendations are only as good as the occupancy data. Because occupancy is entered by administrators, delays or errors will degrade results. Timestamps, staleness flags and periodic confirmation prompts are planned mitigations; automated check-in (for example QR-based) is a possible future step.

Connectivity. A web application depends on internet access, which disasters may disrupt. Progressive web app caching, SMS delivery and store-and-forward approaches such as the delay-tolerant network of [11] are directions for improving resilience.

Decision support only. The platform recommends; it does not command. Authorized personnel keep final authority, and the ranking rules and weights are visible and adjustable so that they can be inspected.

Privacy and fairness. Location and contact data are sensitive and are minimized. Accessibility and facility criteria are included so that people with special needs are not disadvantaged, and the effect of the weights on such users will be examined during evaluation.

Herdin. If many users see the same recommendation at the same moment, that shelter can be overloaded. The transactional capacity re-check and the spare-capacity term reduce this risk, and it will be tested explicitly with simultaneous-request scenarios.

IX. CONCLUSION AND FUTURE WORK

We presented the design of the Emergency Shelter Allocation Platform, a web-based, capacity-aware decision-support system that centralizes shelter information and recommends suitable shelters with route and capacity details. The design links six key problems to platform features, uses a layered open-source architecture, and ranks shelters through a staged pipeline of availability, capacity, suitability, distance and weighted scoring, with an occupancy feedback loop intended to reduce overcrowding. A review of eleven related works indicates that an end-user-facing service of this kind is a useful complement to existing planning models.

Future work includes completing the implementation, running the experiments of Section VII to report quantitative results, and extending the platform with batch allocation for many simultaneous requests using integer programming as in [11]; route-risk awareness as in [8]; learned demand and suitability estimates as in [4], [10]; and offline and multilingual access.

ACKNOWLEDGMENT

The authors thank Irfan Rajab Bhatt, Associate Professor, School of Computer Science and Engineering, Presidency University, for supervising this project.

REFERENCES

- [1] K. Yang *et al.*, “Intelligent shelter allotment for emergency evacuation planning: A case study of Makkah,” *IEEE Intell. Syst.*, vol. 30, no. 5, pp. 66–76, Sep./Oct. 2015, doi: 10.1109/MIS.2015.39.
- [2] J. Zhang, M. Zhang, F. Ren, and J. Liu, “An innovation approach for optimal resource allocation in emergency management,” *IEEE Trans. Comput.*, 2016, doi: 10.1109/TC.2016.2601331.
- [3] F. Wang, Z. Pei, L. Dong, and J. Ma, “Emergency resource allocation for multi-period post-disaster using multi-objective cellular genetic algorithm,” *IEEE Access*, vol. 8, pp. 82255–82265, 2020, doi: 10.1109/ACCESS.2020.2991865.
- [4] A. Amiran, B. Behnam, and S. Seyedin, “Machine learning for earthquake emergency evacuation: Site selection and neighborhood navigation,” *IEEE Access*, vol. 13, pp. 91595–91610, 2025, doi: 10.1109/ACCESS.2025.3573207.
- [5] D. Sarma, A. Das, U. K. Bera, and A. Singh, “A cost minimization resource allocation model for disaster relief operations with an information crowdsourcing-based MCDM approach,” *IEEE Trans. Eng. Manag.*, vol. 69, no. 5, pp. 2454–2474, 2022, doi: 10.1109/TEM.2020.3015775.
- [6] D. Li, Y. Li, and L. Wang, “Model and algorithms for emergency service facility location problem,” in *Proc. 2009 IITA Int. Conf. Services Sci., Manag. Eng. (SSME)*, 2009, pp. 15–18, doi: 10.1109/SSME.2009.80.
- [7] L. Tanti, S. Efendi, M. S. Lydia, and H. Mawengkang, “Designing an optimization model for dynamic facility-location problem at post-disaster area considering uncertainty,” in *Proc. 4th Int. Conf. Cybern. Intell. Syst. (ICORIS)*, 2022, pp. 1–6, doi: 10.1109/ICORIS56080.2022.10031495.
- [8] S. Cui, S. Liu, X. Tang, and T. Zhu, “Emergency material allocation problem considering post-disaster impact,” in *Proc. 8th Int. Conf. Ind. Technol. Manag. (ICITM)*, 2019, pp. 290–294, doi: 10.1109/ICITM.2019.8710729.
- [9] O. Rodríguez-Espíndola, “Multi-period model for disaster management in simultaneous disasters,” *IEEE Trans. Eng. Manag.*, vol. 71, pp. 4971–4984, 2024, doi: 10.1109/TEM.2022.3227329.

- [10] B. Rajalakshmi, P. Aswini, H. P. Thethi, H. R. Goyal, J. Sravanthi, and A. A. Hameed, "Utilizing artificial intelligence for efficient resource allocation and logistics in humanitarian aid," in *Proc. 1st Int. Conf. Sustain. Comput. Integr. Commun. Changing Landscape AI (ICSCAI)*, 2024, pp. 1–6, doi: 10.1109/ICSCAI61790.2024.10866107.
- [11] S. Basu, S. Roy, S. Bandyopadhyay, and S. Das Bit, "A utility driven post disaster emergency resource allocation system using DTN," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 50, no. 7, pp. 2338–2350, Jul. 2020, doi: 10.1109/TSMC.2018.2813008.
- [12] OpenStreetMap contributors. "OpenStreetMap." [Online]. Available: <https://www.openstreetmap.org> (Accessed: Sep. 24, 2026).
- [13] "Leaflet: An open-source JavaScript library for interactive maps." [Online]. Available: <https://leafletjs.com> (Accessed: Sep. 24, 2026).
- [14] "PostGIS: Spatial and geographic objects for PostgreSQL." [Online]. Available: <https://postgis.net> (Accessed: Sep. 24, 2026).
- [15] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Eng. Task Force, RFC 7519, May 2015.
- [16] R. Jain, D.-M. Chiu, and W. R. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corp., Hudson, MA, USA, Tech. Rep. DEC-TR-301, Sep. 1984.